Best case analysis

- #compares in merge step : 1

1 2 3 4 5 6

$3 \le 4 \Rightarrow$ already merged

- Recurrence relation :

$C(n) :=$ #compares of merge sort on size n array

$$C(n) = 2C(n/2) + 1 \qquad C(1) = 0$$

$$= 2[2C(n/4) + 1] + 1$$

$$= 4C(\tfrac{n}{4}) + 2 + 1$$

$$= 4[2C(\tfrac{n}{8}) + 1] + 2 + 1$$

$$= 8C(\tfrac{n}{8}) + 4 + 2 + 1$$

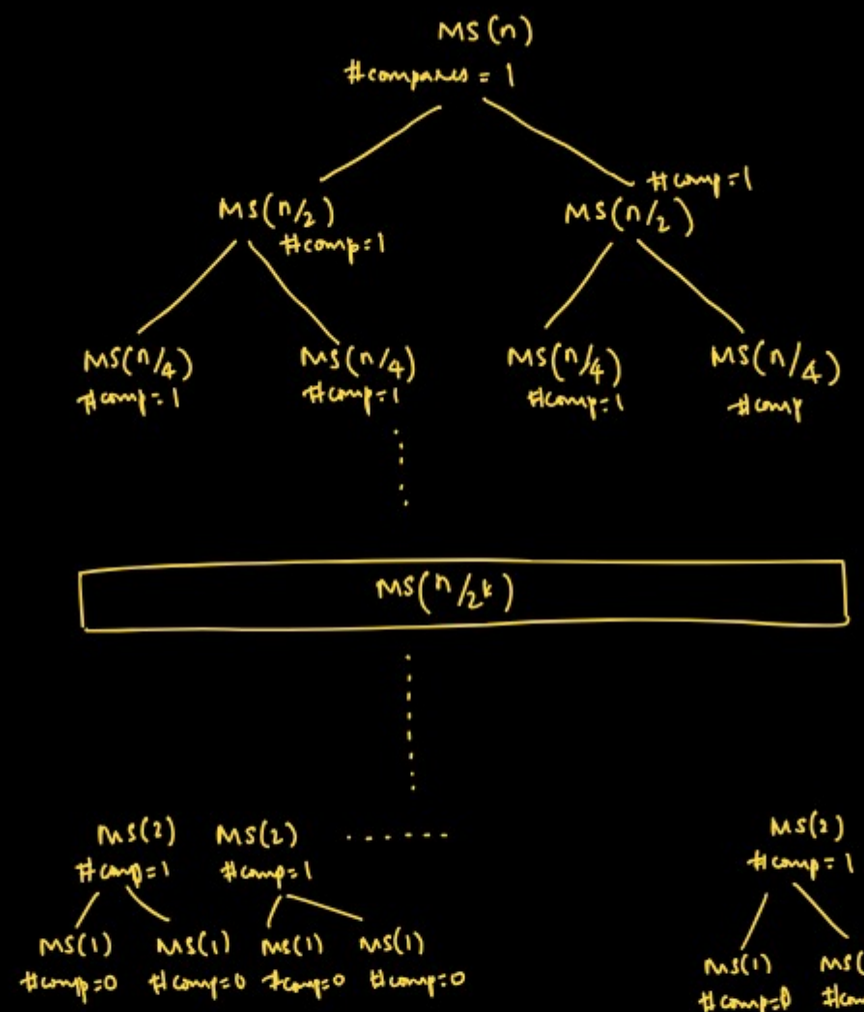
$$= 8[2C(\tfrac{n}{16}) + 1] + 4 + 2 + 1$$

$$= 16C(\tfrac{n}{16}) + 8 + 4 + 2 + 1$$

$$\vdots$$

$$= kC(1) + \underbrace{\ldots + 8 + 4 + 2 + 1}_{\log_2 n \text{ terms}}$$

$$\underbrace{1 + 2 + 4 + 8 + \ldots}_{\underbrace{2^0 + 2^1 + 2^2 + 2^3 + \ldots}_{m = \log_2 n \text{ terms}}} = a\frac{[2^m - 1]}{2-1} = 2^{\log_2 n} - 1 = n - 1 = \Theta(n)$$



Level	#calls	#compares/call	#total compares
0	1	1	1
1	2	1	2
2	4	1	4
⋮	⋮	⋮	⋮
k	2^k	1	2^k
⋮	⋮	⋮	⋮
$\log_2 n - 1$	$2^{\log_2 n - 1} = \frac{2^{\log_2 n}}{2^1} = \frac{n}{2}$	1	$2^{\log_2 n - 1}$ or $(n/2)$
$\log_2 n$	$n = 2^{\log_2 n}$	0	0

#total compares

$$= \underbrace{1 + 2 + 4 + \cdots + 2^k + \cdots + 2^{\log_2 n - 1}}_{\log_2 n \text{ terms}}$$

$$= \underbrace{2^0 + 2^1 + 2^2 + \cdots\cdots}_{m = \log_2 n \text{ terms}} = a\frac{[2^m - 1]}{2-1} = 1\frac{[2^{\log_2 n} - 1]}{1} = n - 1 = \Theta(n)$$

Comparable

```
class Student implements Comparable<Student> {
    int id;
    String name;
    int age;
    public static final Comparator<Student> BY_NAME = new NameComparator();
    public static final Comparator<Student> BY_AGE = new AgeComparator();
    public static final Comparator<Student> BY_AGE_REV = new RevAgeComparator();

    public int compareTo(Student that) {
        if (this.id < that.id) return -1;
        if (this.id > that.id) return +1;
        return 0;
    }

    private static class NameComparator implements Comparator<Student> {
        public int compare(Student s1, Student s2) {
            return s1.name.compareTo(s2.name);
        }
    }

    private static class AgeComparator implements Comparator<Student> {
        public int compare(Student s1, Student s2) {
            return s1.age - s2.age;
        }
    }

    private static class RevAgeComparator implements Comparator<Student> {
        public int compare(Student s1, Student s2) {
            return -(s1.age - s2.age);
        }
    }
}
```

```
Student[] students = {...};
Arrays.sort(students);
Arrays.sort(students, Student.BY_NAME);
Arrays.sort(students, Student.BY_AGE);
Arrays.sort(students, Student.BY_AGE_REV);
```

- In place sorting (definition)
  - A sorting algo is inplace if amount of extra memory is $\leq c.\log n = O(\log n)$ → Note: We have to include the space taken for recursion
  - Selection sort : Memory $= \Theta(1) \Rightarrow$ Inplace
  - Insertion sort : Extra memory : $\Theta(1) \Rightarrow$ Inplace
  - Bubble sort : Extra memory : $\Theta(1) \Rightarrow$ Inplace
  - Merge sort :
    - Memory used for recursion : $\Theta(\log n)$
    - Aux array : $\Theta(n)$

Total extra memory $= \Theta(n) + \Theta(\log n)$
$= \Theta(n)$
$\Rightarrow$ Not inplace

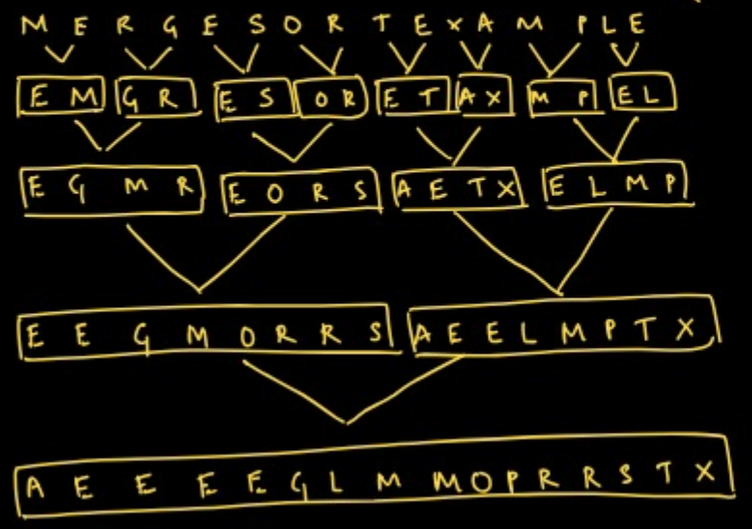
→ max #stack frames that are there at any point on the stack = height of the recursion tree

Space per stack frame = 'k' bytes

max #stack frames : ht of recursion tree $= \log_2 n + 1$

Total extra space for recursion $= (\log_2 n + 1)\,k = \Theta(\log n)$

Bottom up merge sort : Note: Space complexity is still $\Theta(n)$ (for aux array)

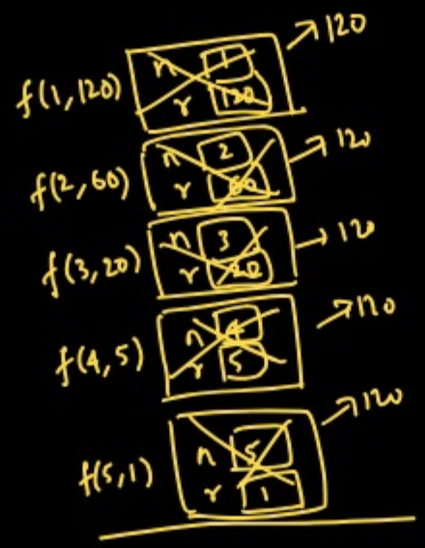


```
int factorial (int n) {
    if (n ≤ 1) return 1;
    return n * factorial (n-1);
}
```

$0! = 1$, $1! = 1$, $2! = 2$

```
int factorial (int n, int result) {
    if (n ≤ 1) return result;
    return factorial (n-1, n*result);
}

//wrapper
int factorial(int n) {
    return factorial (n, 1);
}
```



Tail recursion

Inversions: Inversion is a pair of elements that are out of order

2 4 3 5 3 1 4 3

- #inversions:

$2 \rightarrow (2,1)$

$4 \rightarrow (4,3)\ (4,3)\ (4,1)\ (4,3)$

$3 \rightarrow (3,1)$

$5 \rightarrow (5,3)\ (5,1)\ (5,4)\ (5,3)$

$3 \rightarrow (3,1)$

$4 \rightarrow (4,3)$

$\underline{12}$

$x_1 \quad x_2 \quad x_3 \quad x_4 \quad x_5$

$x_1 \rightarrow x_2\ x_3\ x_4\ x_5$

$x_2 \rightarrow x_3\ x_4\ x_5$

$x_3 \rightarrow x_4\ x_5$

$x_4 \rightarrow x_5$

$x \le y$ $\qquad$ $!(x > y)$

$x=2$	$y=3$ ✓		$x=2$	$y=3$	✗
$x=2$	$y=2$ ✓		$x=2$	$y=2$	✗

1. $x < y \iff y > x$
2. $x \le y \iff y \ge x$
3. $x > y \iff y < x$
4. $x \ge y \iff y \le x$

not (x is less than y)
$\equiv$ x is not less than y
$\equiv$ x is greater than equal to y

- $!(x < y) \iff x \ge y$
- $!(x \le y) \iff x > y$
- $!(x > y) \iff x \le y$
- $!(x \ge y) \iff x < y$

5. $x \ge y \iff !(x < y)$
6. $x > y \iff !(x \le y)$
7. $x \le y \iff !(x > y)$
8. $x < y \iff !(x \ge y)$

x greater than or equal to y

x is not less than y

left = 0 ⇒ # inner loop executions = $(n-1)$

left = 1 $\quad = n-2$

left = 2 $\quad = n-3$

$\vdots$

left = $n-2$ $\quad = 1$

$$\underbrace{[1+2+\cdots+(n-1)]}_{\frac{(n-1)\times n}{2}}\underbrace{\Theta(1)}_{\text{if condition}} = \Theta(n^2)$$

• # max inversions in an array of length 'n':

6 5 4 3 2 1

(6,5) (5,4) (2,1)
(6,4) (5,3) .....
(6,3) (5,2)
(6,2) (5,1)
(6,1)

inversions : $n-1 + n-2 + \cdots + 1 + 0 = \frac{n(n-1)}{2}$

$\boxed{2}$ 4 3 5 3 1 4 3

$\boxed{2\ 4}$ 3 5 3 1 4 3 (4,3)

$\boxed{2\ 3\ 4}$ 5 3 1 4 3

$\boxed{2\ 3\ 4\ 5}$ 3 1 4 3 (4,3) (5,3)

$\boxed{2\ 3\ 3\ 4\ 5}$ 1 4 3 (2,1) (3,1) (3,1) (4,1) (5,1)

$\boxed{1\ 2\ 3\ 3\ 4\ 5}$ 4 3 (5,4)

$\boxed{1\ 2\ 3\ 3\ 4\ 4\ 5}$ 3 (4,3) (4,3), (5,3)

- Counting #inversions recursively

```
int sortAndCountInv (int[] array) {
    n = array.length
    if(n==1) return 0;

    left, right = split(array);
    leftInversions = sortAndCountInv (left);
    rightInversions = sortAndCountInv (right);
    splitInversions = mergeAndCountSplitInv (left, right)

    return leftInversions + rightInversions
           + splitInversions;
}
```

$T(n)$ - time for sortAndCountInv on array of length 'n'

$T(n) = 2T(n/2) + \underline{kn}$

$\downarrow$

$T(n) = \Theta(n \log n)$

3	8	1	3

5	4	2	3

left inv: (3,1)
(8,1)
(8,3)

right inv: (5,4)
(5,2)
(5,3)
(4,2)
(4,3)

split inversions: (3,2)
(8,5)
(8,4)
(8,2)
(8,3)
(3,2)

Two ideas:

1. Sorting left & right halves individually doesn't change the #split inversions.

1	3	3	8

2	3	4	5

split inversions: (3,2)
(3,2)
(8,2)
(8,3)
(8,4)
(8,5)

2. Merge step uncovers split inversions

1	3	3	8

2	3	4	5

1	2	3	3	3	4	5	

#split: 6

Split inv:
+1 (3,2)
+1 (3,2)
+4 (8,2)
(8,3)
(8,4)
(8,5)

left[i] ≤ right[j] ⇒ we move left[i] into the merged part

left[i] > all elements in right[0:j-1]
left[i] ≤ all elements in right[j:m]

split inversions corresponding to left[i]: (left[i], right[0])
(left[i], right[1])
⋮
(left[i], right[j-1])

```
sum=0;  int array[n];
for(i=0; i<n ; i++) {
    int e: ary[i];
    sum: sum + e;
}
```

Init: i=0 1 step

for each i:

1. i<n 1 step
2. int e ; 1 step
3. ary(i); 1 step
4. e = ary(i); 1 step
5. sum + e ; 1 step
6. sum: res; 1 step
7. i++ 1 step

n times

n=i<n 1 step

Total RT: $1 + 7n + 1$
$= \underline{\underline{7n+2}}$

Quicksort

- runs with average time of $n \log n$
- worst could go upto n^2
- very fast in practice. typically 30-40% faster than merge sort.
- randomized algorithm.

- Partition (crux of quick sort)

0	1	2	3	4	5	6	7	8
O	I	K	Q	T	C	S	U	R

pivot

0	1	2	3	4	5	6	7	8
I	K	C	O	Q	T	S	U	R

elements smaller than pivot | O | elements larger than pivot

(after partition)

pivot

goes to its place in the sorted

pivot goes where it needs to go after sorting

```
quicksort(arr):
    shuffle(arr)
    sort(arr, 0, arr.length-1)

sort(arr, lo, hi)
    pivot = arr[lo]
    pivot_index = partition(arr, lo, hi)
    sort(lo, pivot_index - 1)
    sort(pivot_index + 1, hi)
```

0	1	2	3	4	5	6	7	8
O	I	K	Q	T	C	S	U	R

sort(0, 8)

3 = pi = partition(0, 8)

0	1	2	3	4	5	6	7	8
I	K	C	O	S	T	Q	R	U

sort(0, 2)

1 = pi: partition(0,2)

0	1	2	3					
C	I	K	O	S	T	Q	R	U

sort(0, 0) → BC

sort(2, 2) → BC

sort(4, 8)

6 = pi: partition(4,8)

						6		
C	I	K	O	R	Q	S	T	U

sort(4,5)

5 = pi = partition(4,5)

				4	5	6		
C	I	K	O	Q	R	S	T	U

sort(4,4) → BC

sort(6,5) → BC No right half so lo > hi

sort(7,8)

7 = pi: partition(7,8)

							7	
C	I	K	O	Q	R	S	T	U

sort(7,6) → BC (no left half)

sort(8,8) → BC

	-5	0	3	16	17	22	200	360
pivot	17	360	22	0	-5	3	16	200

0	-5	3	16	17	200	22	360

↑↑

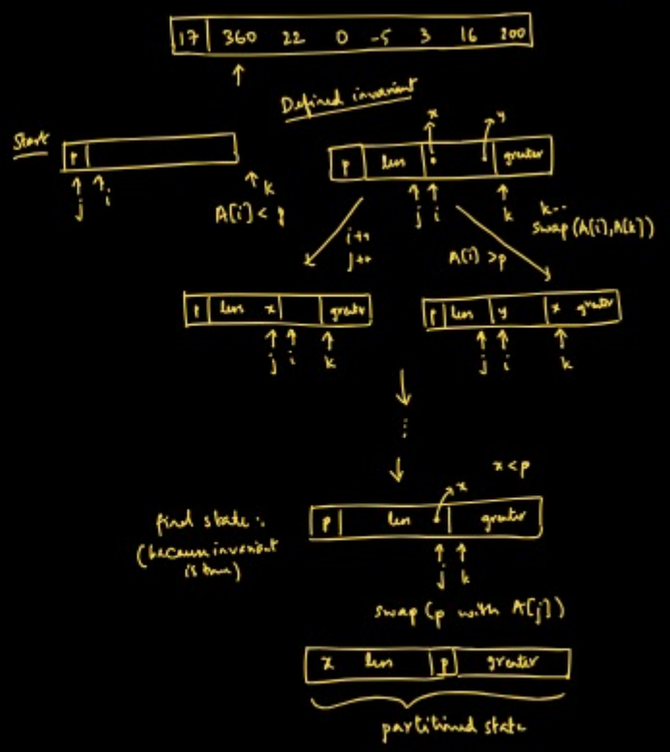
less	p	greater		→ ····· →	less	p	greater

↑j ↑i

Partition inplace

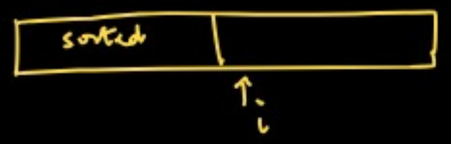
0	1	2	3				
0	-5	3	16	17	360	22	200

↑



Insertion sort

Invariant:



Base case: start state ✓

Transition: (one step of the outerloop)

we are moving A[i] to the left until we find a smaller or equal element

→ Invariant is proved at this point

final state

- n^2+n+41 is a prime

$$\begin{cases} n=1 & ✓ \\ n=2 & ✓ \\ n=3 & ✓ \\ \vdots & \\ n=39 & ✓ \\ n=40 & ✗ \text{ as } 40^2+40+41 \text{ not a prime} \end{cases}$$

⇒ Proof by example doesn't work

$$S(n) = 1+2+3+\dots+n$$

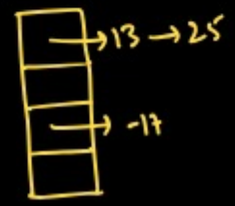
$$S(n) = \frac{n(n+1)}{2}$$

$$S(2) = 1+2 = 3 = \frac{2\times 3}{2} = 3$$

$$S(5) = 1+2+3+4+5 = 15 = \frac{5\times 6}{2} = 15$$

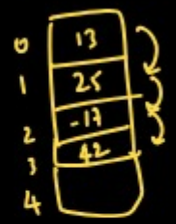
Separate chaining

4 buckets



$h(13) = 0$
$h(-17) = 2$
$h(25) = 0$

Linear probing



$h(13) = 0$
$h(-17) = 2$
$h(25) = 0$
$h(42) = 0$